JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class ScheduledThreadPoolExecutor

java.lang.Object
  java.util.concurrent.AbstractExecutorService
    java.util.concurrent.ThreadPoolExecutor
      java.util.concurrent.ScheduledThreadPoolExecutor

All Implemented Interfaces:

AutoCloseable, Executor, ExecutorService, ScheduledExecutorService

---

```
public class ScheduledThreadPoolExecutor
extends ThreadPoolExecutor
implements ScheduledExecutorService
```

A `ThreadPoolExecutor` that can additionally schedule commands to run after a given delay, or to execute periodically. This class is preferable to `Timer` when multiple worker threads are needed, or when the additional flexibility or capabilities of `ThreadPoolExecutor` (which this class extends) are required.

Delayed tasks execute no sooner than they are enabled for execution, but without any real-time guarantees about when, after they are enabled, they will commence. Tasks scheduled for exactly the same execution time are enabled in first-in-first-out (FIFO) order of submission.

When a submitted task is cancelled before it is run, execution is suppressed. By default, such a cancelled task is not automatically removed from the work queue until its delay elapses. While this enables further inspection and monitoring, it may also cause unbounded retention of cancelled tasks. To avoid this, use `setRemoveOnCancelPolicy(boolean)` to cause tasks to be immediately removed from the work queue at time of cancellation.

Successive executions of a periodic task scheduled via `scheduleAtFixedRate` or `scheduleWithFixedDelay` do not overlap. While different executions may be performed by different threads, the effects of prior executions *happen-before* those of subsequent ones.

While this class inherits from `ThreadPoolExecutor`, a few of the inherited tuning methods are not useful for it. In particular, because it acts as a fixed-sized pool using `corePoolSize` threads and an unbounded queue, adjustments to `maximumPoolSize` have no useful effect. Additionally, it is almost never a good idea to set `corePoolSize` to zero or use `allowCoreThreadTimeOut` because this may leave the pool without threads to handle tasks once they become eligible to run.

As with `ThreadPoolExecutor`, if not otherwise specified, this class uses `Executors.defaultThreadFactory()` as the default thread factory, and `ThreadPoolExecutor.AbortPolicy` as the default rejected execution handler.

Extension notes: This class overrides the `execute` and `submit` methods to generate internal `ScheduledFuture` objects to control per-task delays and scheduling. To preserve functionality, any further overrides of these methods in subclasses must invoke superclass versions, which effectively disables additional task customization. However, this class provides alternative protected extension method `decorateTask` (one version each for `Runnable` and `Callable`) that can be used to customize the concrete task types used to execute commands entered via `execute`, `submit`, `schedule`, `scheduleAtFixedRate`, and `scheduleWithFixedDelay`. By default, a `ScheduledThreadPoolExecutor` uses a task type extending `FutureTask`. However, this may be modified or replaced using subclasses of the form:

```
public class CustomScheduledExecutor extends ScheduledThreadPoolExecutor {

  static class CustomTask<V> implements RunnableScheduledFuture<V> { ... }

  protected <V> RunnableScheduledFuture<V> decorateTask(
               Runnable r, RunnableScheduledFuture<V> task) {
      return new CustomTask<V>(r, task);
  }

  protected <V> RunnableScheduledFuture<V> decorateTask(
               Callable<V> c, RunnableScheduledFuture<V> task) {
      return new CustomTask<V>(c, task);
  }
  // ... add constructors, etc.
}
```

Since:

1.5

Nested Class Summary

Nested classes/interfaces declared in class ThreadPoolExecutor
`ThreadPoolExecutor.AbortPolicy`, `ThreadPoolExecutor.CallerRunsPolicy`, `ThreadPoolExecutor.DiscardOldestPolicy`, `ThreadPoolExecutor.DiscardPolicy`

Constructor Summary

Constructors

Constructor	Description

ScheduledThreadPoolExecutor(int corePoolSize)	Creates a new ScheduledThreadPoolExecutor with the given core pool size.
ScheduledThreadPoolExecutor(int corePoolSize, RejectedExecutionHandler handler)	Creates a new ScheduledThreadPoolExecutor with the given initial parameters.
ScheduledThreadPoolExecutor(int corePoolSize, ThreadFactory threadFactory)	Creates a new ScheduledThreadPoolExecutor with the given initial parameters.
ScheduledThreadPoolExecutor(int corePoolSize, ThreadFactory threadFactory, RejectedExecutionHandler handler)	Creates a new ScheduledThreadPoolExecutor with the given initial parameters.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
protected <V> RunnableScheduled<V>	**decorateTask**(Runnable runnable, RunnableScheduledFuture<	Modifies or replaces the task used to execute a runnable.
protected <V> RunnableScheduled<V>	**decorateTask**(Callable<V> callable, RunnableScheduledFuture<	Modifies or replaces the task used to execute a callable.
void	**execute**(Runnable command)	Executes command with zero required delay.
boolean	**getContinueExistingPeriod**	Gets the policy on whether to continue executing existing periodic tasks even when this executor has been shutdown.
boolean	**getExecuteExistingDelaye**	Gets the policy on whether to execute existing delayed tasks even when this executor has been shutdown.

BlockingQueue<Runnable>	**getQueue**()	Returns the task queue used by this executor.
boolean	**getRemoveOnCancelPolicy**(	Gets the policy on whether cancelled tasks should be immediately removed from the work queue at time of cancellation.
ScheduledFuture<?>	**schedule**(Runnable command, long delay, TimeUnit unit)	Submits a one-shot task that becomes enabled for execution after the given delay.
<V> ScheduledFuture<V>	**schedule**(Callable<V> callable, long delay, TimeUnit unit)	Submits a value-returning one-shot task that becomes enabled for execution after the given delay.
ScheduledFuture<?>	**scheduleAtFixedRate**(Runnable command, long initialDelay, long period, TimeUnit unit)	Submits a periodic action that becomes enabled for execution first after the given initial delay, and subsequently with the given period; that is, executions will commence after `initialDelay`, then `initialDelay + period`, then `initialDelay + 2 * period`, and so on.
ScheduledFuture<?>	**scheduleWithFixedDelay**(Runnable command, long initialDelay, long delay, TimeUnit unit)	Submits a periodic action that becomes enabled for execution first after the given initial delay, and subsequently with the given delay between the termination of one execution and the

		commencement of the next.
void	**setContinueExistingPerio**(boolean value)	Sets the policy on whether to continue executing existing periodic tasks even when this executor has been `shutdown`.
void	**setExecuteExistingDelaye**(boolean value)	Sets the policy on whether to execute existing delayed tasks even when this executor has been `shutdown`.
void	**setRemoveOnCancelPolicy**(boolean value)	Sets the policy on whether cancelled tasks should be immediately removed from the work queue at time of cancellation.
void	**shutdown**()	Initiates an orderly shutdown in which previously submitted tasks are executed, but no new tasks will be accepted.
List<Runnable>	**shutdownNow**()	Attempts to stop all actively executing tasks, halts the processing of waiting tasks, and returns a list of the tasks

		that were awaiting execution.
Future<?>	**submit**(Runnable task)	Submits a Runnable task for execution and returns a Future representing that task.
<T> Future<T>	**submit**(Runnable task, T result)	Submits a Runnable task for execution and returns a Future representing that task.
<T> Future<T>	**submit** (Callable<T> task)	Submits a value-returning task for execution and returns a Future representing the pending results of the task.

Methods declared in class ThreadPoolExecutor

afterExecute, allowCoreThreadTimeOut, allowsCoreThreadTimeOut, awaitTermination, beforeExecute, getActiveCount, getCompletedTaskCount, getCorePoolSize, getKeepAliveTime, getLargestPoolSize, getMaximumPoolSize, getPoolSize, getRejectedExecutionHandler, getTaskCount, getThreadFactory, isShutdown, isTerminated, isTerminating, prestartAllCoreThreads, prestartCoreThread, purge, remove, setCorePoolSize, setKeepAliveTime, setMaximumPoolSize, setRejectedExecutionHandler, setThreadFactory, terminated, toString

Methods declared in class AbstractExecutorService

invokeAll, invokeAll, invokeAny, invokeAny, newTaskFor, newTaskFor

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Methods declared in interface ExecutorService

close

Constructor Details

ScheduledThreadPoolExecutor

```
public ScheduledThreadPoolExecutor(int corePoolSize)
```

Creates a new `ScheduledThreadPoolExecutor` with the given core pool size.

Parameters:

`corePoolSize` - the number of threads to keep in the pool, even if they are idle, unless `allowCoreThreadTimeOut` is set

Throws:

`IllegalArgumentException` - if `corePoolSize < 0`

ScheduledThreadPoolExecutor

```
public ScheduledThreadPoolExecutor(int corePoolSize,
                                   ThreadFactory threadFactory)
```

Creates a new `ScheduledThreadPoolExecutor` with the given initial parameters.

Parameters:

`corePoolSize` - the number of threads to keep in the pool, even if they are idle, unless `allowCoreThreadTimeOut` is set

`threadFactory` - the factory to use when the executor creates a new thread

Throws:

`IllegalArgumentException` - if `corePoolSize < 0`

`NullPointerException` - if `threadFactory` is null

ScheduledThreadPoolExecutor

```
public ScheduledThreadPoolExecutor(int corePoolSize,
                                   RejectedExecutionHandler handler)
```

Creates a new `ScheduledThreadPoolExecutor` with the given initial parameters.

Parameters:

`corePoolSize` - the number of threads to keep in the pool, even if they are idle, unless `allowCoreThreadTimeOut` is set

`handler` - the handler to use when execution is blocked because the thread bounds and queue capacities are reached

Throws:

`IllegalArgumentException` - if `corePoolSize < 0`

`NullPointerException` - if `handler` is null

ScheduledThreadPoolExecutor

```
public ScheduledThreadPoolExecutor(int corePoolSize,
                                   ThreadFactory threadFactory,
                                   RejectedExecutionHandler handler)
```

Creates a new `ScheduledThreadPoolExecutor` with the given initial parameters.

Parameters:

`corePoolSize` - the number of threads to keep in the pool, even if they are idle, unless `allowCoreThreadTimeOut` is set

`threadFactory` - the factory to use when the executor creates a new thread

`handler` - the handler to use when execution is blocked because the thread bounds and queue capacities are reached

Throws:

`IllegalArgumentException` - if `corePoolSize < 0`

`NullPointerException` - if `threadFactory` or `handler` is null

Method Details

decorateTask

```
protected <V> RunnableScheduledFuture<V> decorateTask
(Runnable runnable,
 RunnableScheduledFuture<V> task)
```

Modifies or replaces the task used to execute a runnable. This method can be used to override the concrete class used for managing internal tasks. The default implementation simply returns the given task.

Type Parameters:

`V` - the type of the task's result

Parameters:

`runnable` - the submitted Runnable

`task` - the task created to execute the runnable

Returns:

a task that can execute the runnable

Since:

1.6

decorateTask

```
protected <V> RunnableScheduledFuture<V> decorateTask
(Callable<V> callable,
 RunnableScheduledFuture<V> task)
```

Modifies or replaces the task used to execute a callable. This method can be used to override the concrete class used for managing internal tasks. The default implementation simply returns the given task.

Type Parameters:

`V` - the type of the task's result

Parameters:

`callable` - the submitted Callable

`task` - the task created to execute the callable

Returns:

a task that can execute the callable

Since:

1.6

schedule

```
public ScheduledFuture<?> schedule(Runnable command,
                                   long delay,
                                   TimeUnit unit)
```

Description copied from interface: `ScheduledExecutorService`

Submits a one-shot task that becomes enabled for execution after the given delay.

Specified by:

`schedule` in interface `ScheduledExecutorService`

Parameters:

`command` - the task to execute

`delay` - the time from now to delay execution

`unit` - the time unit of the delay parameter

Returns:

a ScheduledFuture representing pending completion of the task and whose `get()` method will return `null` upon completion

Throws:

`RejectedExecutionException` - if the task cannot be scheduled for execution

`NullPointerException` - if command or unit is null

schedule

```
public <V> ScheduledFuture<V> schedule(Callable<V> callable,
                                       long delay,
                                       TimeUnit unit)
```

Description copied from interface: `ScheduledExecutorService`

Submits a value-returning one-shot task that becomes enabled for execution after the given delay.

Specified by:

`schedule` in interface `ScheduledExecutorService`

Type Parameters:

`V` - the type of the callable's result

Parameters:

`callable` - the function to execute

`delay` - the time from now to delay execution

`unit` - the time unit of the delay parameter

Returns:

a ScheduledFuture that can be used to extract result or cancel

Throws:

`RejectedExecutionException` - if the task cannot be scheduled for execution

`NullPointerException` - if callable or unit is null

scheduleAtFixedRate

```
public ScheduledFuture<?> scheduleAtFixedRate(Runnable command,
                                              long initialDelay,
                                              long period,
                                              TimeUnit unit)
```

Submits a periodic action that becomes enabled for execution first after the given initial delay, and subsequently with the given period; that is, executions will commence after `initialDelay`, then `initialDelay + period`, then `initialDelay + 2 * period`, and so on.

The sequence of task executions continues indefinitely until one of the following exceptional completions occur:

- The task is explicitly cancelled via the returned future.
- Method `shutdown()` is called and the policy on whether to continue after shutdown is not set true, or method `shutdownNow()` is called; also resulting in task cancellation.
- An execution of the task throws an exception. In this case calling `get` on the returned future will throw `ExecutionException`, holding the exception as its cause.

Subsequent executions are suppressed. Subsequent calls to `isDone()` on the returned future will return `true`.

If any execution of this task takes longer than its period, then subsequent executions may start late, but will not concurrently execute.

Specified by:

`scheduleAtFixedRate` in interface `ScheduledExecutorService`

Parameters:

`command` - the task to execute

`initialDelay` - the time to delay first execution

`period` - the period between successive executions

`unit` - the time unit of the initialDelay and period parameters

Returns:

a ScheduledFuture representing pending completion of the series of repeated tasks. The future's `get()` method will never return normally, and will throw an exception upon task cancellation or abnormal termination of a task execution.

Throws:

`RejectedExecutionException` - if the task cannot be scheduled for execution

`NullPointerException` - if command or unit is null

`IllegalArgumentException` - if period less than or equal to zero

scheduleWithFixedDelay

```
public ScheduledFuture<?> scheduleWithFixedDelay(Runnable command,
                                                 long initialDelay,
                                                 long delay,
                                                 TimeUnit unit)
```

Submits a periodic action that becomes enabled for execution first after the given initial delay, and subsequently with the given delay between the termination of one execution and the commencement of the next.

The sequence of task executions continues indefinitely until one of the following exceptional completions occur:

- The task is explicitly cancelled via the returned future.
- Method `shutdown()` is called and the policy on whether to continue after shutdown is not set true, or method `shutdownNow()` is called; also resulting in task cancellation.
- An execution of the task throws an exception. In this case calling `get` on the returned future will throw `ExecutionException`, holding the exception as its cause.

Subsequent executions are suppressed. Subsequent calls to `isDone()` on the returned future will return `true`.

Specified by:

`scheduleWithFixedDelay` in interface `ScheduledExecutorService`

Parameters:

`command` - the task to execute

`initialDelay` - the time to delay first execution

`delay` - the delay between the termination of one execution and the commencement of the next

`unit` - the time unit of the initialDelay and delay parameters

Returns:

a ScheduledFuture representing pending completion of the series of repeated tasks. The future's `get()` method will never return normally, and will throw an exception upon task cancellation or abnormal termination of a task execution.

Throws:

RejectedExecutionException - if the task cannot be scheduled for execution

NullPointerException - if command or unit is null

IllegalArgumentException - if delay less than or equal to zero

execute

```
public void execute(Runnable command)
```

Executes command with zero required delay. This has effect equivalent to schedule(command, 0, anyUnit). Note that inspections of the queue and of the list returned by shutdownNow will access the zero-delayed ScheduledFuture, not the command itself.

A consequence of the use of ScheduledFuture objects is that afterExecute is always called with a null second Throwable argument, even if the command terminated abruptly. Instead, the Throwable thrown by such a task can be obtained via Future.get().

Specified by:

execute in interface Executor

Overrides:

execute in class ThreadPoolExecutor

Parameters:

command - the task to execute

Throws:

RejectedExecutionException - at discretion of RejectedExecutionHandler, if the task cannot be accepted for execution because the executor has been shut down

NullPointerException - if command is null

submit

```
public Future<?> submit(Runnable task)
```

Description copied from interface: ExecutorService

Submits a Runnable task for execution and returns a Future representing that task. The Future's get method will return null upon *successful* completion.

Specified by:

submit in interface ExecutorService

Overrides:

submit in class AbstractExecutorService

Parameters:

`task` - the task to submit

Returns:

a Future representing pending completion of the task

Throws:

`RejectedExecutionException` - if the task cannot be scheduled for execution

`NullPointerException` - if the task is null

submit

```
public <T> Future<T> submit(Runnable task,
                            T result)
```

Description copied from interface: `ExecutorService`

Submits a Runnable task for execution and returns a Future representing that task. The Future's get method will return the given result upon successful completion.

Specified by:

`submit` in interface `ExecutorService`

Overrides:

`submit` in class `AbstractExecutorService`

Type Parameters:

`T` - the type of the result

Parameters:

`task` - the task to submit

`result` - the result to return

Returns:

a Future representing pending completion of the task

Throws:

`RejectedExecutionException` - if the task cannot be scheduled for execution

`NullPointerException` - if the task is null

submit

```
public <T> Future<T> submit(Callable<T> task)
```

Description copied from interface: `ExecutorService`

Submits a value-returning task for execution and returns a Future representing the pending results of the task. The Future's `get` method will return the task's result upon successful completion.

If you would like to immediately block waiting for a task, you can use constructions of the form `result = exec.submit(aCallable).get();`

Note: The `Executors` class includes a set of methods that can convert some other common closure-like objects, for example, `PrivilegedAction` to `Callable` form so they can be submitted.

Specified by:

`submit` in interface `ExecutorService`

Overrides:

`submit` in class `AbstractExecutorService`

Type Parameters:

T - the type of the task's result

Parameters:

`task` - the task to submit

Returns:

a Future representing pending completion of the task

Throws:

`RejectedExecutionException` - if the task cannot be scheduled for execution

`NullPointerException` - if the task is null

setContinueExistingPeriodicTasksAfterShutdownPolicy

```
public void setContinueExistingPeriodicTasksAfterShutdownPolicy
(boolean value)
```

Sets the policy on whether to continue executing existing periodic tasks even when this executor has been `shutdown`. In this case, executions will continue until `shutdownNow` or the policy is set to `false` when already shutdown. This value is by default `false`.

Parameters:

`value` - if `true`, continue after shutdown, else don't

See Also:

`getContinueExistingPeriodicTasksAfterShutdownPolicy()`

getContinueExistingPeriodicTasksAfterShutdownPolicy

```
public boolean getContinueExistingPeriodicTasksAfterShutdownPolicy()
```

Gets the policy on whether to continue executing existing periodic tasks even when this executor has been `shutdown`. In this case, executions will continue until `shutdownNow` or the policy is set to `false` when already shutdown. This value is by default `false`.

Returns:

`true` if will continue after shutdown

See Also:

`setContinueExistingPeriodicTasksAfterShutdownPolicy(boolean)`

setExecuteExistingDelayedTasksAfterShutdownPolicy

```
public void setExecuteExistingDelayedTasksAfterShutdownPolicy
(boolean value)
```

Sets the policy on whether to execute existing delayed tasks even when this executor has been `shutdown`.
In this case, these tasks will only terminate upon `shutdownNow`, or after setting the policy to `false` when already shutdown. This value is by default `true`.

Parameters:

`value` - if `true`, execute after shutdown, else don't

See Also:

`getExecuteExistingDelayedTasksAfterShutdownPolicy()`

getExecuteExistingDelayedTasksAfterShutdownPolicy

```
public boolean getExecuteExistingDelayedTasksAfterShutdownPolicy()
```

Gets the policy on whether to execute existing delayed tasks even when this executor has been `shutdown`.
In this case, these tasks will only terminate upon `shutdownNow`, or after setting the policy to `false` when already shutdown. This value is by default `true`.

Returns:

`true` if will execute after shutdown

See Also:

`setExecuteExistingDelayedTasksAfterShutdownPolicy(boolean)`

setRemoveOnCancelPolicy

```
public void setRemoveOnCancelPolicy(boolean value)
```

Sets the policy on whether cancelled tasks should be immediately removed from the work queue at time of cancellation. This value is by default `false`.

Parameters:

`value` - if `true`, remove on cancellation, else don't

Since:

1.7

See Also:

`getRemoveOnCancelPolicy()`

getRemoveOnCancelPolicy

```
public boolean getRemoveOnCancelPolicy()
```

Gets the policy on whether cancelled tasks should be immediately removed from the work queue at time of cancellation. This value is by default `false`.

Returns:

`true` if cancelled tasks are immediately removed from the queue

Since:

1.7

See Also:

`setRemoveOnCancelPolicy(boolean)`

shutdown

```
public void shutdown()
```

Initiates an orderly shutdown in which previously submitted tasks are executed, but no new tasks will be accepted. Invocation has no additional effect if already shut down.

This method does not wait for previously submitted tasks to complete execution. Use `awaitTermination` to do that.

If the `ExecuteExistingDelayedTasksAfterShutdownPolicy` has been set `false`, existing delayed tasks whose delays have not yet elapsed are cancelled. And unless the `ContinueExistingPeriodicTasksAfterShutdownPolicy` has been set `true`, future executions of existing periodic tasks will be cancelled.

Specified by:

`shutdown` in interface `ExecutorService`

Overrides:

`shutdown` in class `ThreadPoolExecutor`

shutdownNow

`public List<Runnable> shutdownNow()`

Attempts to stop all actively executing tasks, halts the processing of waiting tasks, and returns a list of the tasks that were awaiting execution. These tasks are drained (removed) from the task queue upon return from this method.

This method does not wait for actively executing tasks to terminate. Use `awaitTermination` to do that.

There are no guarantees beyond best-effort attempts to stop processing actively executing tasks. This implementation interrupts tasks via `Thread.interrupt()`; any task that fails to respond to interrupts may never terminate.

Specified by:

`shutdownNow` in interface `ExecutorService`

Overrides:

`shutdownNow` in class `ThreadPoolExecutor`

Returns:

list of tasks that never commenced execution. Each element of this list is a `ScheduledFuture`. For tasks submitted via one of the `schedule` methods, the element will be identical to the returned `ScheduledFuture`. For tasks submitted using `execute`, the element will be a zero-delay `ScheduledFuture`.

getQueue

`public BlockingQueue<Runnable> getQueue()`

Returns the task queue used by this executor. Access to the task queue is intended primarily for debugging and monitoring. This queue may be in active use.
Retrieving the task queue does not prevent queued tasks from executing.

Each element of this queue is a `ScheduledFuture`. For tasks submitted via one of the `schedule` methods, the element will be identical to the returned `ScheduledFuture`. For tasks submitted using `execute`, the element will be a zero-delay `ScheduledFuture`.

Iteration over this queue is *not* guaranteed to traverse tasks in the order in which they will execute.

Overrides:

`getQueue` in class `ThreadPoolExecutor`

Returns:

the task queue

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.